

面试手册

数据结构

把基础，读成底气。

1 个科目 / 20 道问题

回答要点 • 常见追问 • 易错点 • 参考来源

整理日期：2026-10-09

本手册总结经典面试知识点，不是全网公司面经的穷举，也不代表真实招聘频次统计。题解为独立中文整理，具体版本与实现请参考题目中的原始资料。

C++ 以 C++17/20 为主；MySQL 以 8.4 LTS InnoDB 为主；Redis 以开源版常用功能为主。目录、PDF 书签及参考链接均可点击。

阅读目录

阅读目录	2
数据结构	3
01. 时间复杂度、空间复杂度与摊还分析怎么区分？	3
02. 数组与链表怎么选？链表插入真的是 $O(1)$ 吗？	3
03. 栈、队列、循环队列与双端队列有什么区别？	4
04. 哈希表为什么平均 $O(1)$ ，却可能退化？	5
05. 二分查找如何避免边界错误？能对答案二分吗？	6
06. 常见排序算法怎样比较？稳定性有什么意义？	6
07. 快速排序如何分区？为什么会退化，怎样优化？	7
08. 归并排序为何稳定？如何统计逆序对？	8
09. 二叉堆为何能 $O(n)$ 建堆？Top K 怎么做？	9
10. BST、AVL 与红黑树有什么区别？	9
11. 二叉树遍历与重建的条件是什么？	10
12. Trie 如何支持前缀搜索？空间开销怎样优化？	11
13. 图用邻接矩阵还是邻接表？BFS 与 DFS 如何选？	11
14. 拓扑排序怎样检测有向环？顺序是否唯一？	12
15. BFS、Dijkstra、Bellman-Ford 与 Floyd 怎么选择？	13
16. 最小生成树与最短路不同在哪里？Prim 和 Kruskal 如何工作？	13
17. 并查集的路径压缩与按秩合并为什么有效？	14
18. 动态规划的状态、转移、边界与遍历顺序怎么设计？	15
19. 单调栈、双指针与滑动窗口为什么能做到线性？	16
20. 怎样设计平均 $O(1)$ 的 LRU 缓存？	16

数据结构

从复杂度到结构与算法选择

20 题覆盖线性结构、查找排序、树图与常见算法模式。复杂度默认采用常见 RAM 模型，并写明平均、最坏和摊还条件；实现常数、内存与缓存表现仍需具体测量。

01. 时间复杂度、空间复杂度与摊还分析怎么区分？

复杂度与线性结构 / 基础

复杂度描述资源消耗随输入规模的增长，不能直接等同于某次运行耗时。最坏、平均和摊还分析回答不同问题；空间分析还要说明是否计入输入、输出以及递归调用栈。

回答要点

1. O 给出渐进上界， Θ 描述紧确增长量级；分析嵌套循环时应累加真实执行次数，不应看到两层循环就一律判成平方。若内层范围随外层变化，应先写出求和式再求增长阶。
2. 平均复杂度依赖输入或随机过程的概率假设，摊还复杂度分析一串操作的总体成本，不要求概率分布。动态数组倍增时，少数线性迁移的成本可分摊到很多尾部追加操作中。
3. 空间复杂度通常关注额外空间，但必须明确口径。递归深度 h 常产生 $O(h)$ 调用栈；输出一个含 n 个元素的答案至少需要相应输出空间，不能一边省略它一边声称完整任务只占常数空间。

常见追问

追问：动态数组追加为什么不是每次 $O(1)$ ？答：容量充足时可以常数完成，容量不足时要分配并迁移已有元素，单次可能 $O(n)$ 。按几何倍率扩容的一串 n 次追加总成本可为 $O(n)$ ，因此每次摊还 $O(1)$ ，这不是单次最坏保证。

易错点

没有输入分布就随意说“平均很快”并不严谨。还要区分数据规模、键长度和输出量等变量，字符串哈希不能总按与字符数无关的常数操作处理。

延伸阅读 / 原始参考资料

Open Data Structures • 动态数组与摊还成本

Princeton Algorithms • 复杂度速查

02. 数组与链表怎么选？链表插入真的是 $O(1)$ 吗？

复杂度与线性结构 / 基础

数组支持常数随机访问但中间插删要搬移，链表通过节点链接便于已知位置的结构调整，却需要线性查找。链表 $O(1)$ 插入删除通常以已经获得适当节点或前驱为前提。

回答要点

1. 连续数组可按下标直接定位，遍历通常有较好的缓存局部性；中间插入或保序删除需要搬移后续元素。若不要求保持顺序，有时可用尾元素覆盖删除位置，但会改变排列语义。
2. 链表第 k 个节点一般要从头或尾逐步查找。单链表删除给定节点时通常需要前驱，双链表有前后指针可在节点已知时直接调整；节点分配与额外指针也增加内存及常数开销。
3. 选择结构时先列出访问模式：随机读取多倾向数组，频繁在已知节点旁操作且要求节点地址稳定时可考虑链表。不能只比较插删复杂度，忽略定位、缓存、分配与维护引用的成本。

常见追问

追问：只有单链表待删节点指针能 $O(1)$ 删除吗？答：可在存在后继且允许改变节点内容时，将后继内容搬到当前节点再删后继，但这改变节点身份，尾节点也无法这样处理。若身份或外部引用有要求，就仍需要前驱信息。

易错点

链表的常数删除前提要说完整：已知位置、足够的相邻链接信息以及允许的节点语义。用一次搬移数值的技巧代替一般链表删除会漏掉尾节点和身份约束。

延伸阅读 / 原始参考资料

Open Data Structures • 链表取舍

Open Data Structures • 双向链表

03. 栈、队列、循环队列与双端队列有什么区别？

复杂度与线性结构 / 基础

栈按后进先出工作，队列按先进先出工作，双端队列允许两端操作。循环队列利用取模复用数组空间，关键是清楚定义头尾指针含义，以及如何区分满和空。

回答要点

1. 栈适合括号匹配、表达式处理和深度优先过程；队列适合广度优先搜索与按到达顺序处理。抽象数据类型约束的是操作与顺序，实现可以使用数组或链表，而不是某一种固定布局。
2. 循环数组队列让 `head` 指向待取元素，`tail` 指向下一写入位置是一种常见约定。可以额外记录 `size`，也可以留一个空槽区分满和空；容量计算和判满公式必须与选定约定一致。
3. 双端队列支持头尾插入删除，适合维护滑动窗口候选项。数组实现要处理环绕与扩容，链表实现要维护两端链接；接口复杂度还取决于固定容量、扩容策略和是否需要随机访问。

常见追问

追问：如何用两个栈实现队列？答：入队压入输入栈，出队在输出栈为空时把输入栈所有元素倒过去，再弹出输出栈顶。一次倒栈可能线性，但每个元素只经历有限次转移，整串操作的单次成本为摊还 $O(1)$ 。

易错点

不要混用尾指向“最后一个元素”和“下一写入位置”两种约定，否则容易少算容量或覆盖数据。并发队列还需要额外同步，环形结构本身不保证线程安全。

延伸阅读 / 原始参考资料

Open Data Structures • 数组队列

Open Data Structures • 数组双端队列

04. 哈希表为什么平均 $O(1)$ ，却可能退化？

查找与排序 / 基础

哈希表将键映射到桶或探测位置，借助良好散列与受控负载取得平均常数时间操作。碰撞无法完全避免，处理策略、负载因子和键的哈希成本决定实际表现，最坏可线性退化。

回答要点

1. 拉链法把同桶元素放在链表或其他结构中，查询需要在桶内检查键相等。负载因子等于元素数除以桶数；分布均匀且负载受控时，桶内平均工作量有限，但集中碰撞会拉长查询路径。
2. 开放寻址把元素留在桶数组内，碰撞后按照探测序列寻找位置。删除不能随意把槽位标成从未使用，否则会截断后续查询链；通常使用墓碑或调整探测簇，并适时清理与扩容。
3. 扩容重哈希需要重新布置大量元素，单次可能线性；按合适增长策略可分摊维护成本。字符串或组合键的哈希计算随键长度增长，自定义哈希还必须保证相等键具有相同哈希值。

常见追问

追问：碰撞是不是说明哈希函数错了？答：不是，有限输出范围映射更大键空间，碰撞通常不可避免。错误在于分布长期很差，或相等键被算出不同哈希导致查询失效。工程上要同时检查等价规则、负载和攻击性输入。

易错点

哈希相同不表示键相等，仍要用等价比较确认。平均常数结论依赖分布与负载条件；不能把一次重哈希或恶意碰撞的最坏延迟藏在 $O(1)$ 说法里。

延伸阅读 / 原始参考资料

Open Data Structures • 拉链哈希

Open Data Structures • 线性探测

05. 二分查找如何避免边界错误？能对答案二分吗？

查找与排序 / 基础

二分查找利用有序或单调条件反复缩小候选区间。先定义区间开闭和循环不变量，再统一更新边界；寻找第一个满足条件的位置时，返回值可能等于区间末尾，必须检查。

回答要点

1. 采用半开区间 $[left, right)$ 查第一个不小于目标的元素时， $left$ 左侧已知更小， $right$ 及其右侧已知不小。中点不满足则令 $left=mid+1$ ，满足则令 $right=mid$ ，直到边界相遇。
2. 中点通常写成 $left+(right-left)/2$ ，避免直接相加造成溢出；仍需选用能表示区间长度的类型。空数组、单元素、重复值、目标在两端之外，都应按照同一不变量得到明确结果。
3. 答案二分要求判定函数存在单调转折，例如容量足够后继续增大仍可完成任务。总复杂度是判定成本乘二分轮数；不能只说 $O(\log n)$ ，也不能对没有单调性的业务条件直接套用。

```
#include <vector>
#include <cstdint>
std::size_t lower(const std::vector<int>& a, int x) {
    std::size_t l = 0, r = a.size();
    while (l < r) {
        auto m = l + (r - l) / 2;
        if (a[m] < x) l = m + 1;
        else r = m;
    }
    return l;
}
```

常见追问

追问：二分找到目标后为什么还可能继续查？答：存在重复元素时，普通等值查找只保证找到某个位置。要找最左或最右边界，需要在满足条件时继续缩小相应一侧，或分别使用 `lower_bound` 与 `upper_bound`，形成重复值范围。

易错点

比较次数对数不等于所有容器上的总时间对数；链表定位中点可能需要线性移动。必须说明有随机访问支持，或算法能直接利用搜索树结构。

延伸阅读 / 原始参考资料

Princeton Algorithms • 二分查找

ISO C++ 工作草案 • 二分算法

06. 常见排序算法怎样比较？稳定性有什么意义？

查找与排序 / 基础

比较排序通常以 $O(n \log n)$ 为通用高效目标；选择时还要看最坏时间、额外空间、稳定性与输入特征。稳定排序保持相等键的原始相对顺序，在多关键字处理和记录排序中很有用。

回答要点

1. 插入排序最坏 $O(n^2)$ ，但对近乎有序的小数据常有效，常见实现可以稳定；选择排序通常 $O(n^2)$ 且不稳定。冒泡排序也常为平方时间，是否早停影响有序输入上的最佳表现。
2. 常见数组归并排序稳定且最坏 $O(n \log n)$ ，代价是 $O(n)$ 辅助数组；堆排序最坏 $O(n \log n)$ ，常见实现额外空间常数但不稳定。普通快速排序平均高效，最坏仍可能平方且不稳定。
3. 计数与基数排序利用整数范围或位数等额外结构，不能用来否定比较模型下的 $\Omega(n \log n)$ 下界。计数排序复杂度通常含键范围 K ，范围巨大时即便元素很少也可能付出很大空间成本。

常见追问

追问：怎样按两个键排序并保留顺序？答：可以直接提供字典序比较器；也可以先按次要键排序，再按主要键进行稳定排序，让主要键相等的记录保留次要键顺序。若第二次排序不稳定，之前的次序就可能丢失。

易错点

“原地”是否允许递归栈要明确，不要把快排的分区辅助空间与整个递归空间混为一谈。排序性能还受比较器成本、缓存与数据分布影响。

延伸阅读 / 原始参考资料

Princeton Algorithms • 排序特征

Open Data Structures • 计数与基数排序

07. 快速排序如何分区？为什么会退化，怎样优化？

查找与排序 / 进阶

快速排序选择枢轴，把元素按与枢轴的关系分区，再处理两侧。分区越均衡递归越浅，平均可达 $O(n \log n)$ ；若反复出现极不平衡分区，时间会退化为 $O(n^2)$ 。

回答要点

1. 先选定一种分区定义，例如枢轴最终归位，或双指针返回切分边界；递归范围必须与该方案一致。不能将 Hoare 分区返回值当作另一种方案中的最终枢轴位置，容易重复或漏掉元素。
2. 每次选最左元素会在特定有序输入上得到极不平衡分区。随机枢轴或先打乱输入可改善期望表现，但不提供普通随机快排的确切最坏上界；三数取中同样不能根除全部坏输入。
3. 大量重复值适合三路分区，将小于、等于和大于枢轴的区域分开，减少对相等元素的重复递归。小区间可用插入排序；需要确定上界时可用达到深度阈值后切换堆排的混合策略。

常见追问

追问：快排空间复杂度一定 $O(\log n)$ 吗？答：常规递归在均衡或平均情况下栈深对数，极端分区时可能线性。每次先递归较小分区、迭代处理较大分区，可限制递归栈深度，但这不自动消除总时间平方退化。

易错点

把随机化说成最坏 $O(n \log n)$ 是常见错误。还要测试重复值和边界相遇，保证每次分区后子问题严格缩小，避免死循环。

延伸阅读 / 原始参考资料

Princeton Algorithms • 快速排序

Open Data Structures • 分区与比较排序

08. 归并排序为何稳定？如何统计逆序对？

查找与排序 / 进阶

归并排序把问题分成两个子区间，分别排序后线性合并。合并相等键时先取左侧元素可保持稳定性；统计逆序对时，右元素小于当前左元素便与剩余左区间形成逆序。

回答要点

1. 两个已排序区间可以用双指针合并，每次取较小元素，任何一侧用尽后拷贝另一侧剩余部分。分治递推 $T(n)=2T(n/2)+O(n)$ ，得到最坏 $O(n \log n)$ ，不依赖初始是否有序。
2. 普通数组实现通常需要 $O(n)$ 合并缓冲区，递归还使用 $O(\log n)$ 栈。稳定性依赖相等元素的取舍规则，若相等时总先取右侧，就可能改变来自左右半区的原始相对次序。
3. 当右侧当前元素严格小于左侧当前元素时，它小于左半区剩余所有元素，可一次增加剩余数量。先递归统计两半内部逆序，再统计跨区间逆序，总计仍可在线性合并的基础上完成。

常见追问

追问：逆序对结果为什么要用宽整数？答： n 个元素最多有 $n(n-1)/2$ 个逆序对，规模稍大就可能超过 32 位整数。计算乘法时也要先转换到足够宽的类型，不能先在窄类型中溢出，再把错误结果转成宽类型。

易错点

逆序定义通常使用严格大于，相等值不应计数。稳定合并时使用左侧优先的相等规则，可同时让逆序统计条件清晰，避免把重复值算进答案。

延伸阅读 / 原始参考资料

Princeton Algorithms • 归并排序

Princeton 官方代码 • Merge

09. 二叉堆为何能 $O(n)$ 建堆？Top K 怎么做？

树与索引结构 / 基础

二叉堆以完全二叉树结构维持父子优先级关系，取堆顶 $O(1)$ ，插入和删除堆顶 $O(\log n)$ 。自底向上建堆为 $O(n)$ ，并不是对每个节点都按完整树高付费。

回答要点

1. 数组下标从零开始时，节点 i 的孩子为 $2i+1$ 和 $2i+2$ ，非根节点的父亲为 $(i-1)/2$ 。插入后上浮、移除堆顶后将末尾放到根再下沉，保持堆序及完全树的结构。
2. 从最后一个非叶节点向前下沉，绝大多数节点距离叶层很近。按不同高度累计节点数与下沉长度，总成本是线性的；若从空堆逐个插入所有元素，则一般上界为 $O(n \log n)$ 。
3. 求最大的 K 个数可维护大小 K 的小根堆，让当前最弱候选处于堆顶。新元素更大时替换堆顶并下沉，总时间 $O(n \log K)$ 、空间 $O(K)$ ；若需要有序输出，还需对结果再排序。

常见追问

追问：为什么不能在堆里 $O(\log n)$ 找任意值？答：堆序只约束父子，不保证整个数组有序，也不能根据一个节点值排除另一半区域。查任意值通常仍需 $O(n)$ ；若要高效修改指定元素，可额外维护键到位置的索引并同步交换。

易错点

“堆中元素有序”不准确，只有堆顶极值及父子关系得到保证。 K 为零、 K 大于输入量、重复值是否占名额和输出是否排序，均应先明确。

延伸阅读 / 原始参考资料

Open Data Structures • 二叉堆

Princeton Algorithms • 优先队列与堆

10. BST、AVL 与红黑树有什么区别？

树与索引结构 / 进阶

BST 依靠有序键关系缩小查找范围，但普通 BST 可能退化成链。AVL 约束子树高度差，红黑树通过颜色与黑高约束限制高度；二者都能提供对数级查找与更新上界。

回答要点

1. 普通 BST 的操作时间取决于树高 h ，顺序插入可能产生高度 n 的树。删除有两个孩子的节点时，可用中序后继或前驱替换，再处理较简单的实际删除位置，同时维护关联数据。
2. AVL 要求每个节点左右子树高度差至多一，通过单旋或双旋修复不平衡。旋转改变局部父子关系但保留中序键顺序；删除可能需要沿祖先路径继续修复，不能照搬插入停止条件。

3. 红黑树用红节点限制与相同黑高等规则保证高度为 $O(\log n)$ ，更新涉及旋转和重着色。AVL 更紧的高度控制与红黑树不同的修复策略形成取舍，不能不经测量断言某一种对所有负载更快。

常见追问

追问：有序 map 一定就是红黑树吗？答：红黑树是多种标准库的常见实现，但接口规范通常约束复杂度、顺序和迭代器保证，不强制内部结构。面试中应先讲数据结构原理，再明确具体库与实现层面的事实。

易错点

旋转不是重新排序所有节点；它应保留 BST 的中序顺序。重复键规则、父指针更新、根节点替换以及附加统计信息都需要在实现中一致维护。

延伸阅读 / 原始参考资料

Open Data Structures • 二叉搜索树

Open Data Structures • 红黑树

Yale 算法讲义 • Balanced Trees

11. 二叉树遍历与重建的条件是什么？

树与索引结构 / 基础

前序先访问根，中序在左右子树之间访问根，后序最后访问根；层序通过队列逐层处理。不同遍历揭示不同结构信息，重建是否唯一还依赖节点值唯一及给出的遍历组合。

回答要点

1. 前序适合先处理父节点，后序适合先汇总孩子结果，BST 的中序遍历可得到键的有序序列。迭代 DFS 用显式栈保存待处理状态，层序 BFS 用队列保存下一批节点，不能混淆二者访问顺序。
2. 无重复值且序列合法时，前序加中序或后序加中序可唯一重建。用中序值到位置的索引可减少反复查找；每次先确定根，再划分左右子区间，最后验证元素数量与对应位置。
3. 仅有前序和后序通常不能唯一确定普通二叉树，因为一个孩子究竟在左侧还是右侧可能无法辨别。重复值也会让中序根位置有歧义；题目若加满二叉树等限制，结论需要重新分析。

常见追问

追问：DFS 空间一定比 BFS 小吗？答：DFS 常需要 $O(h)$ 路径空间，BFS 常需要 $O(w)$ 层宽空间；链状树 h 可线性，宽树 w 也可线性。选择应结合树形和任务，而不是绝对说一种遍历更省空间。

易错点

区分完全二叉树、满二叉树与完美二叉树，不同教材中文译名可能有差异。重建时应直接说明每个节点孩子数量等结构条件，避免术语误解。

延伸阅读 / 原始参考资料

Open Data Structures • 二叉树遍历

12. Trie 如何支持前缀搜索？空间开销怎样优化？

树与索引结构 / 进阶

Trie 按字符或位逐层组织键，共享公共前缀。查找长度 L 的字符串通常需要 $O(L)$ 路径操作；它适合前缀查询和补全，但空间取决于字符集、节点表示与键的分布。

回答要点

1. 插入和查找逐字符沿边前进，完整键结束处要记录终止标志或值。一个字符串的路径存在不等于该字符串本身已经入库，例如存入 hello 后，hell 可能只是前缀而不是完整键。
2. 前缀检索先定位前缀节点，再遍历其子树收集结果。复杂度必须计入输出数量和输出字符串长度，不能只说前缀长度 $O(L)$ ，因为打印很多候选本身就需要相应时间和空间。
3. 每节点固定字符集数组可快速访问孩子，却浪费稀疏节点空间；映射或紧凑边列表可减少空间但影响访问成本。压缩 Trie 合并只有单个孩子的路径，须额外比较边上的字符片段。

常见追问

追问：Trie 一定比哈希表查字符串快吗？答：两者都可能需要读取字符，哈希表通常先算整个键哈希，Trie 可在不匹配处提前停止并支持前缀操作。实际速度还依赖节点分配、字符集大小、键分布和缓存局部性，应按需求选择。

易错点

必须明确字符编码，不能把 UTF-8 字节数量直接当成用户感知字符数量。固定 26 个孩子只适用于限定字母表，大小写、汉字与其他符号需要另行设计。

延伸阅读 / 原始参考资料

Princeton Algorithms • R-way Trie 官方实现

Princeton Algorithms • Trie 讲义

13. 图用邻接矩阵还是邻接表？BFS 与 DFS 如何选？

图算法 / 基础

邻接矩阵使用 $O(V^2)$ 空间并快速判断某条边，邻接表使用 $O(V+E)$ 空间更适合稀疏图。BFS 按边数逐层扩展，DFS 沿路径深入；遍历复杂度必须结合存储结构说明。

回答要点

1. 邻接矩阵查询某对顶点是否相连常为 $O(1)$ ，但枚举一个顶点的所有邻居通常需扫描 V 个位置。邻接表可直接枚举出边，但判断指定边存在可能需要遍历该顶点邻居或额外索引。

2. 在邻接表上，BFS 与 DFS 完整遍历一般为 $O(V+E)$ 。BFS 对无权图或等权图可找最少边数路径，发现节点时就标记可避免重复入队；DFS 常用于连通分量、环与递归结构分析。
3. 非连通图需对每个尚未访问节点启动遍历，单一起点只能覆盖可达部分。有向图与无向图的连通性、边回退和判环条件不同；无向 DFS 判断环时还要区分返回父边与真正回边。

常见追问

追问：BFS 找到的路径为什么最短？答：队列保证距离为 k 的节点先于距离为 $k+1$ 的节点处理，所以首次发现的边数最少。若边权不一致，这个按层结论不能直接代表总代价最小，应选择适合权重的算法。

易错点

不能把一个 `visited` 标志用于所有有向图判环场景，有向 DFS 还需区分正在递归路径上的节点与已经完成的节点。深图递归 DFS 也可能耗尽调用栈。

延伸阅读 / 原始参考资料

Open Data Structures • 邻接矩阵

Open Data Structures • 图表示与遍历

Princeton Algorithms • 无向图算法

14. 拓扑排序怎样检测有向环？顺序是否唯一？

图算法 / 进阶

拓扑排序将有向边的起点排在终点之前，存在完整拓扑序的充要条件是图为 DAG。Kahn 算法不断输出入度零顶点，若最终输出数小于顶点数，则剩余部分存在有向环。

回答要点

1. 先统计所有顶点入度，将入度为零的顶点入队；弹出一个顶点后遍历它的出边并减少终点入度，新的零入度顶点继续入队。邻接表实现总时间 $O(V+E)$ ，还需 $O(V)$ 辅助状态。
2. 另一种方法是 DFS 完成节点时记入后序，再将后序反转；必须同时检查递归路径中的回边来判环。对有环图仅输出反后序，并不能将它变成满足依赖约束的拓扑排序。
3. 拓扑序可能不唯一，若某一步同时有多个可选零入度顶点，就可能有不同选择。若要求字典序最小，可用小根堆替代普通队列，相应增加维护代价，并明确顶点比较规则。

常见追问

追问：任务依赖怎样建边才不会排反？答：若任务 B 必须等任务 A 完成，建 $A \rightarrow B$ ，B 的入度表示仍未完成的前置条件数量。拓扑排序输出是合法先后序，若还需要完成时间，应结合任务时长沿 DAG 做动态规划。

易错点

输出不到 V 个顶点时不能直接忽略剩余任务，也不能称已经得到完整执行顺序。还应检查自环、孤立顶点和重复边的入度计数是否与存储一致。

延伸阅读 / 原始参考资料

Princeton Algorithms • 拓扑顺序

Princeton Algorithms • DAG 与 Kahn 算法

15. BFS、Dijkstra、Bellman - Ford 与 Floyd 如何选择？

图算法 / 进阶

先看边权与查询范围：无权单源路径用 BFS，非负权常用 Dijkstra，含负边考虑 Bellman - Ford，小规模全点对可用 Floyd；负环会影响有限最短路。

回答要点

1. Dijkstra 每次确定当前暂定距离最小的顶点，并松弛其出边；非负权保证已经确定的距离不会被未来路径降低。采用索引二叉堆和邻接表的常见实现可达 $O((V+E) \log V)$ 时间。
2. Bellman - Ford 通过多轮松弛处理负边，常见最坏 $O(VE)$ ，并可识别源点可达负环。若负环还能到达目标，绕环可无限降低路径代价；与源点不可达的负环不影响该源点查询。
3. Floyd 枚举允许经过的中间顶点，依次更新 $dist[i][j]$ ，通常时间 $O(V^3)$ 、空间 $O(V^2)$ 。更新顺序要让中间顶点循环位于外层；不可达状态相加必须先检查，避免无穷哨兵加法溢出。

常见追问

追问：有负边但没有负环能用 Dijkstra 吗？答：一般不能保证正确，因为负边可能在某顶点已确定后带来更短路径。DAG 可按拓扑序进行线性松弛，即使存在负边也可处理；所以应先查看图结构，而不只根据是否负边机械选择。

易错点

SPFA 等队列优化不能默认承诺线性最坏时间。实现优先队列 Dijkstra 时，若插入重复距离项，还需跳过过期项，并根据具体实现重新说明时间与空间。

延伸阅读 / 原始参考资料

Princeton 讲义 • 单源最短路径

Princeton Algorithms • 负边与负环

16. 最小生成树与最短路不同在哪里？Prim 和 Kruskal 如何工作？

图算法 / 进阶

最小生成树在无向连通加权图中，用无环的 $V - 1$ 条边连接所有顶点并最小化总权重；最短路最小化特定起点到目标的路径代价。两种目标不同，算法结果不能互换。

回答要点

1. Kruskal 按边权从小到大考虑边，连接不同连通分量时才选入，通常借助并查集快速判环。排序常为 $O(E \log E)$ ，随后合并成本较低；图不连通时得到最小生成森林而非一棵生成树。
2. Prim 从一个顶点出发，反复选取当前树到外部的最小权重边来扩张。邻接表加适当二叉堆的常见版本为 $O((V+E) \log V)$ ，稠密图上矩阵版本 $O(V^2)$ 也可能更合适。
3. 割性质支撑贪心正确性：跨越某个割的最小边可成为某棵 MST 的安全选择。负权边不会像最短路负环那样破坏 MST；相同权重可能产生多棵不同的最优树，不能据此说无解。

常见追问

追问：最小生成树一定包含起点到每个节点的最短路径吗？答：不一定，它最小化整个连接网络的总权重，可能让某个点对的树内路径绕远。若需要源点最短路径树，应使用相应最短路算法；若追求建设总代价才考虑 MST。

易错点

不要给有向图直接套普通 Prim 或 Kruskal 求同一意义的 MST。有向的最小树形图是不同问题；边数 $V-1$ 也只是树结构条件，不能单靠条数证明连接且无环。

延伸阅读 / 原始参考资料

Washington CSE373 • 最小生成树讲义

University of São Paulo • Kruskal 官方代码文档

17. 并查集的路径压缩与按秩合并为什么有效？

图算法 / 进阶

并查集维护元素所属集合，find 找代表元，union 合并两个集合。路径压缩减少未来查询路径，按秩或按大小合并避免树过高；联合使用时，一串操作可获得接近常数的摊还成本。

回答要点

1. 每个集合用一棵父指针树表示，根节点作为代表元。判断两个元素是否同属集合，应比较 find 的结果，不能直接比较它们父指针；同一集合中不同节点的直接父亲可能不同。
2. 路径压缩在查询时让沿途节点更接近根，按大小合并把较小树接到较大树上。按秩合并则用秩指导方向，路径压缩后秩不再等于当前真实高度，但仍可用于正确的合并策略。
3. 两种优化联合时常见摊还界为 $O(\alpha(n))$ ， α 是增长极慢的反 Ackermann 函数；这不是每次严格 $O(1)$ 的证明。初始化父数组为 $O(n)$ ，统计集合数时只在成功合并不同根后递减。

```
#include <vector>
int find(std::vector<int>& parent, int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];
    }
}
```

```

    x = parent[x];
}
return x;
}

```

常见追问

追问：并查集能支持删除边吗？答：普通并查集擅长合并与连通查询，不擅长把集合拆开。离线动态连通可考虑倒序操作或回滚并查集；回滚版本通常避免普通路径压缩，以便记录有限的结构改变并恢复。

易错点

union 必须先找到根再修改根关系，直接改任意节点父亲可能破坏集合结构。不要把并查集当作保存原图所有路径信息的结构，它只维护特定连通关系。

延伸阅读 / 原始参考资料

Princeton Algorithms • 并查集复杂度

Princeton Algorithms • 路径压缩与合并

18. 动态规划的状态、转移、边界与遍历顺序怎么设计？

常见算法模式 / 进阶

动态规划复用重叠子问题的答案，核心是定义足够表达后续决策的状态，再建立无环依赖关系。依次说明状态含义、转移、边界、计算顺序、结果位置和复杂度，比先背代码更重要。

回答要点

1. 状态必须用完整句子解释，例如 $dp[i][j]$ 表示两个前缀的最长公共子序列长度。若当前决策还依赖前一个选择、剩余资源或是否持有物品，就需要把相应信息纳入状态，避免不同历史被错误合并。
2. 转移要覆盖所有合法决策，并证明子问题已经包含求解所需信息。最优化问题常需要最优子结构，计数问题则要避免重复计数；记忆化递归与自底向上是处理同一依赖关系的不同计算方式。
3. 复杂度通常等于状态数量乘每个状态的转移成本。只依赖上一行时可用滚动数组压缩空间，但更新顺序必须保持读取的是旧状态；需要还原实际方案时，还要保留选择或前驱信息。

常见追问

追问：0/1 背包一维数组为什么倒序遍历容量？答：每件物品只能使用一次，倒序让 $dp[j-weight]$ 仍来自处理当前物品前的状态。正序更新会读到本轮已更新的状态，可能重复使用同一件物品，更接近完全背包的语义。

易错点

看起来有递推式并不保证状态正确。先验证信息是否足够，以及转移是否重复使用当前对象；空间压缩后测试结果长度正确，也不代表方案还原仍可用。

延伸阅读 / 原始参考资料

19. 单调栈、双指针与滑动窗口为什么能做到线性？

常见算法模式 / 进阶

这些模式通过单向推进或淘汰不会再成为答案的候选，避免重复扫描。单调栈常找相邻更大更小元素，滑动窗口维护连续区间；线性时间需要每个元素进出次数有界及更新成本受控。

回答要点

1. 单调栈维持候选值的单调关系，新元素到来时弹出已经被它解决或支配的候选。寻找下一个更大元素可存索引，遇到更大值便为被弹出的索引确定答案；每个索引最多入栈出栈一次。
2. 滑动窗口常维护区间和、频率或最大值，右边界扩张、左边界按条件收缩。两指针总步数可线性，但前提是不会回退并且状态维护足够高效；条件若缺乏单调性，就不能随意套用。
3. 窗口最大值可用双端队列保留按值递减的候选索引：队首移除过期项，队尾移除被新值支配的项。新值更大且更晚过期，被淘汰的旧值不可能再次成为未来窗口的必要最大候选。

常见追问

追问：求和至少为目标的最短子数组能总用普通滑动窗口吗？答：全部元素非负时，扩张不会减小和，收缩不会增大和，容易保持单调判断。若含负数，这个性质失效，可能需要前缀和结合其他结构，例如单调队列。

易错点

重复值的弹出条件用 $<$ 还是 $\leq$ 会改变保留哪个索引，应按答案要求和过期规则选择。 $O(n)$ 推理要统计整个过程的进出次数，不能只看代码里是否有 `while` 嵌套。

延伸阅读 / 原始参考资料

Cornell CS2110 • 单调栈

Python 官方文档 • deque 双端操作

20. 怎样设计平均 $O(1)$ 的 LRU 缓存？

常见算法模式 / 进阶

LRU 淘汰最久未被使用的项，常用哈希表定位节点、双向链表维护访问次序。命中或更新后移到最近使用端，容量超限从另一端淘汰；平均常数时间依赖哈希表操作的平均复杂度。

回答要点

1. 哈希表保存键到链表节点或迭代器的映射，双向链表从最近访问到最久访问排列。查询命中后把节点摘下并移到头部，指针调整是常数操作；若只用链表，按键定位仍需要线性扫描。

2. 插入新键时先创建节点并更新索引，超过容量就删除尾节点，同时从哈希表移除相同键。更新已有键应改值并移动，而不是生成第二个同键节点；哨兵节点可减少头尾特殊分支。
3. 容量为零、返回值与缺失值冲突、节点所有权和异常路径都要明确。并发访问中命中也会修改访问链，因此所谓只读查询同样可能需要锁；大规模缓存可能采用近似策略减少维护竞争。

常见追问

追问：LRU 和 LFU 有什么区别？答：LRU 看最近访问时间，LFU 看访问频率，两者对一次性扫描与热点的反应不同。LRU 也不自动提供过期时间或内存字节上限，需要将 TTL、容量口径和淘汰策略作为不同需求设计。

易错点

不能只从链表删节点而遗漏哈希索引，否则后续查询会拿到失效地址。严格 $O(1)$ 要避免误写，应说明哈希部分是平均时间，碰撞和重哈希仍可能增加单次成本。

延伸阅读 / 原始参考资料

Oracle Java SE 21 • [LinkedHashMap 访问顺序与 LRU](#)

CMU 论文 • [基于双向链表的 LRU 维护](#)